Filter()

1. Given a list of integers, filter and print only the even numbers.

2. Given a list of names, print only those that start with the letter 'A'.

3. You have a list of Student objects. Filter and print students who scored 60 or above.

4. Given a list of strings, filter out all empty or blank strings.

5. Given a list of products with prices, print only those that cost more than ₹500.

6. Select only passed student from a list.

Map()

1. Given a list of strings, convert each to uppercase and print them.

2. Given a list of integers, square each number and collect the result into a new list.

3. Given a list of integers, square each number and collect the result into a new list.

4. Given a list of names, prefix each name with "Mr." or "Ms." and print the result.

5. Given a list of product prices, apply 10% discount to each and collect the new prices.

6. Multiply by 10 for each element from the list.

7. Add the 5 grace marks to all the failed students (i.e. marks < 35)

Count()

1. get the total number of passed student from the list.

2. Given a list of integers, count how many are even / odd.

3. Given a list of integers, count how many are even.

4. Given a list of Student objects, count how many students passed. (marks >= 40)

5. Convert all words to uppercase and count how many start with "A".

6. Given a list of product prices, apply 20% discount and count how many still cost above ₹500.

Sorted()

1. Given a list of numbers, sort them in increasing order and print them.

2. Sort a list of strings in natural (alphabetical) order.

3. Sort a list of names in reverse (Z → A) order.

4. Sort a list of student names based on the length of their names.

5. Given a list of words, sort them in **ascending order of their length** and print.

6. Given a list of Student objects, sort them by **marks in ascending order**, and print the sorted list.

Min() and max()

1. Given a list of integers, find the **smallest number**.

2. From a list of strings, find the one with the **maximum length**.

3. Given a list of Student objects, find the student who scored the **highest marks**.

4. Filter even numbers from a list and find the **largest even number**.

toArray()

1. Convert a list of strings to a string array.

2. Given an array of integers, use Arrays.stream() to find the sum.

3. Given a string array, convert all elements to uppercase and print.

4. Given a string array, convert it into a list using stream.

Skip()

1. Given a list of names, skip the first 3 names and print the remaining.

2. Given a list of salaries, sort in descending order and skip the top 2 values.

3. Filter even numbers from a list and skip the first 2 elements.

4. Given a list of products, skip the first 2 products priced above ₹500 and print the rest.

5. From a list of words, filter words longer than 3 letters and skip the first 3 of them.

Limit()

1. given a list of student names. Use `limit()` to print only the first 3 names.

2. Given a list of integers, sort them in descending order and print the top 5 values

3. From a list of integers, filter even numbers, limit to the first 4 even numbers, and calculate their total.

4. From a list of words, sort by descending length and display only the first 3 longest words

5. Given a list of `Product` objects, filter products with price > 500, and limit the result to only 2 products.

findFirst()

1. Given a list of names, retrieve the first element.

2. From a list of numbers, get the first even number.

3. Given a list of `Product` objects, find the first product that costs more than ₹500.

4. Given a list of words, return the first word starting with the letter 'J'.

5. Given a list of `Student` objects, find the first student with marks ≥ 40.

anyMatch()

1. Given a list of integers, check if **any number is even**.

2. From a list of names, check if any name starts with the letter 'A'.

3. From a list of students, check if **anyone failed** (marks less than 40).

4. From a list of words, check if any word is entirely in uppercase.

Miscellaneous:

Create class Employee with name, salary, joiningDate and gender.

1. Find the employee who has maximum salary.

2. Find the employee who has second highest salary.

3. Find the employee who is most senior based on joining date.

4. count the employee based on gender.